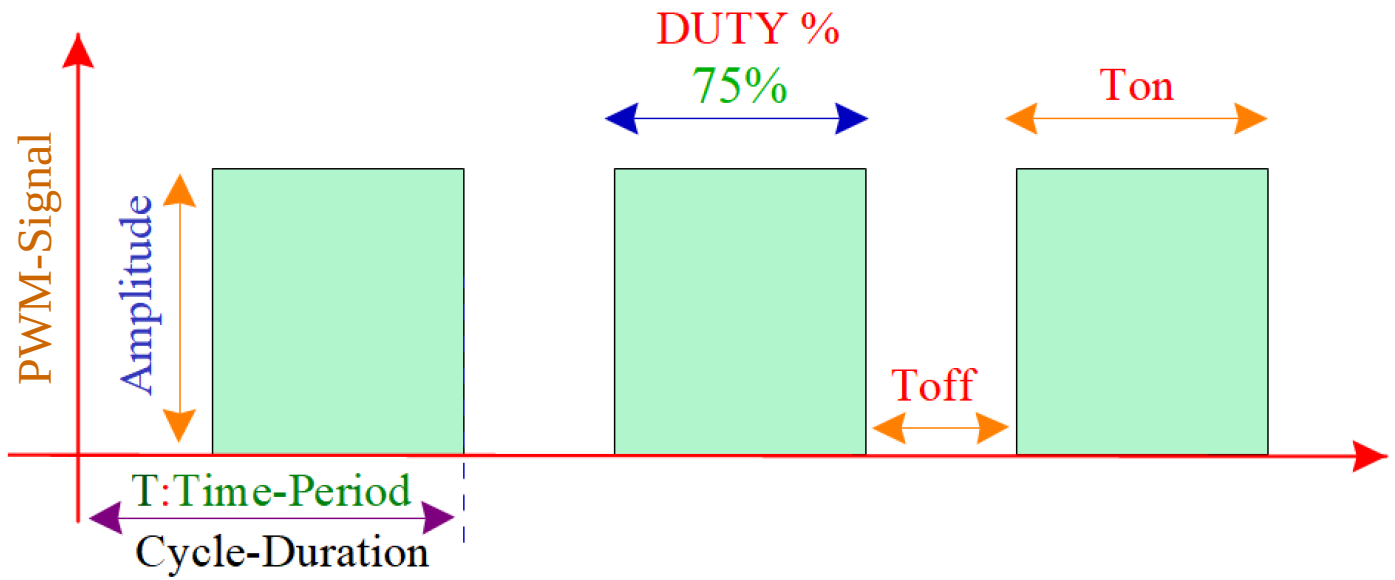
Pulse Width Modulation (PWM)

What is PWM

- PWM generates a periodic square wave signal with a variable Ton
- PWM is a technique used to precisely control analog devices with a digital signal
- PWM signal consists of two main components that define its behavior: a duty cycle and a frequency
- Duty-Cycle describes the amount of time the signal is in a high (on) state as a percentage of the total time to complete one cycle
- Frequency determines how fast the PWM completes a cycle (i.e. 10Hz would be 10 cycles per second)

$$Duty\ Cycle = \frac{T_{ON}}{Cycle\ Duration} \times 100\%$$

The ratio of the pulse width to the cycle duration is called the "duty cycle



- The duty cycle can be used to estimate the apparent analog voltage that a pulse width modulation signal will provide

Output Voltage of PWM

- The higher the duty cycle the higher the average-voltage

$$Average\ Voltage = Duty\ Cycle \times V_{max}$$

Example: Calculate the duty-cycle and the average-voltage for PWM if $T_{on} = 2ms\ , T_{off} = 8ms$ & $V_{max} = 7V$

Solution:

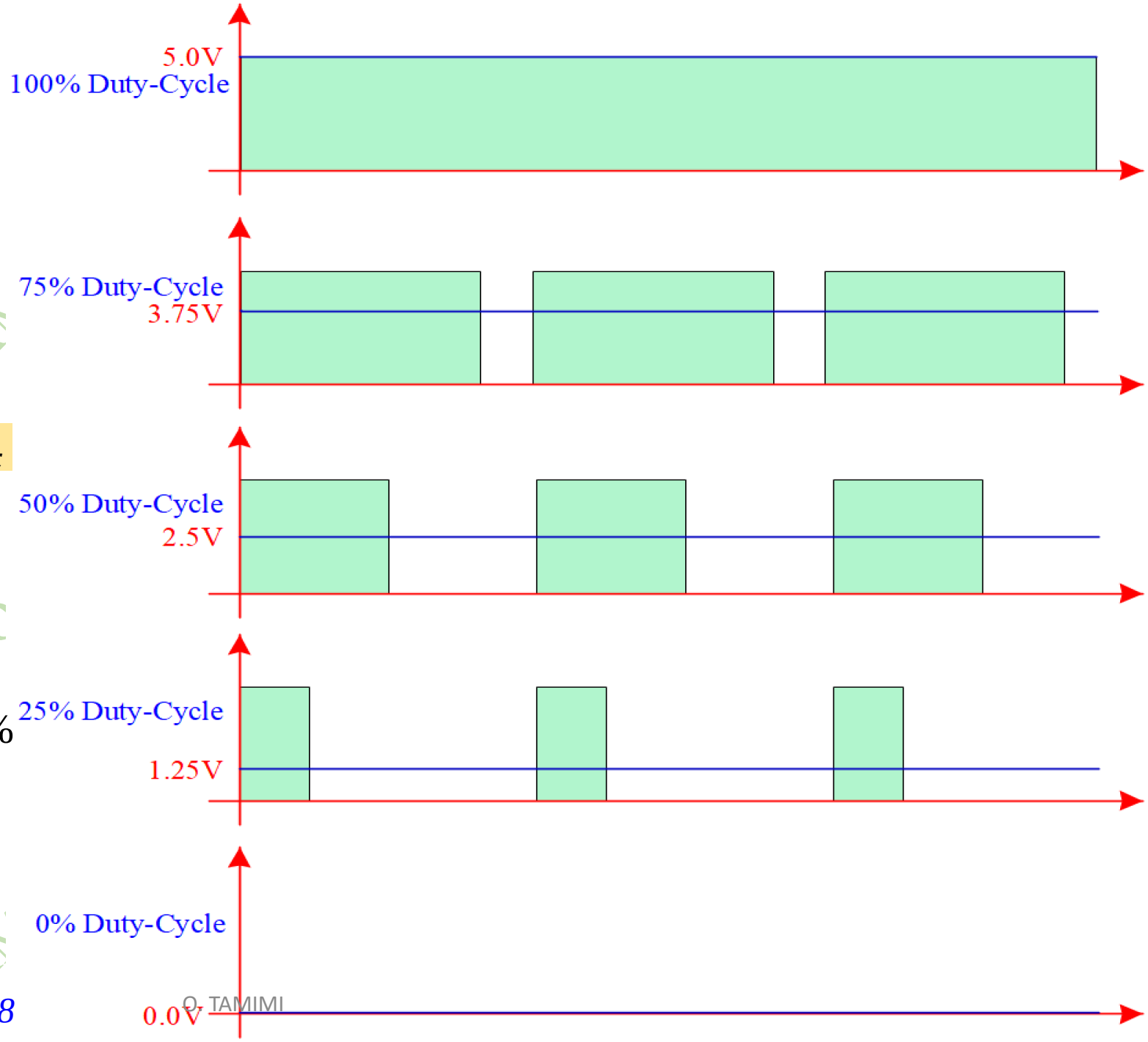
Duty-Cycle=[Ton/Cycle-Duration] x100%

Duty-Cycle=2ms/(2ms+8ms)x100%

Duty-Cycle= 20%

Average-Voltage= (20/100) x 7

Average-Voltage= 1.4 Volt



O. TAMIMI

Why PWM is Important

- PWM is a technique that mitigates the amount of power loss across resistors through electrical loads
- PWM controls the average-current and average-voltage delivered to the load
- This method is accomplished by rapidly turning the switch between the load and the source, on and off

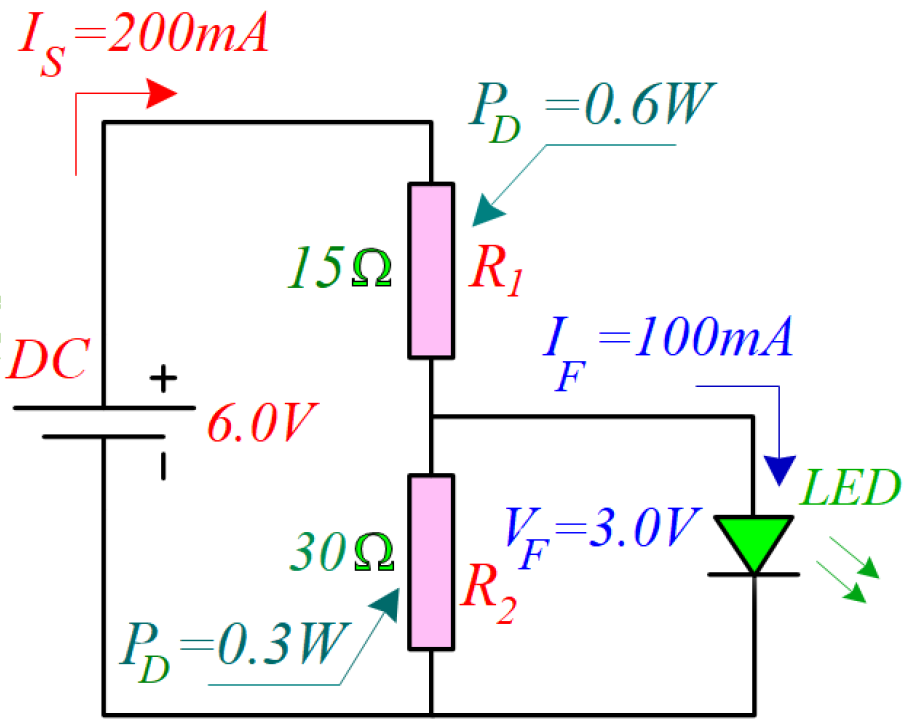
Applications Summary of PWM

- Drive buzzer with different loudness
- Speed control of DC motors
- Adjust the brightness of LCDs & LEDs

Example if you need to drive a power LED with $V_{FD} = 3.0V$ @ $I_F = 100mA$ using a linear DC power supply with 6V

Issues

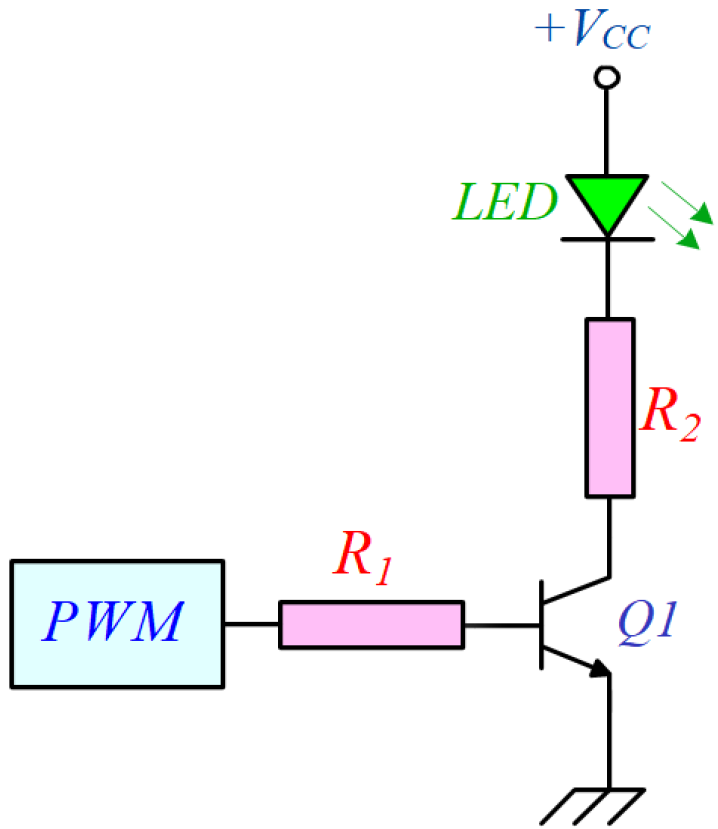
- Bulky power-supply
- High power dissipation across voltage divider resistors R1 & R2
- Half of the source-current is dissipated across resistors



Example if you need to drive the same power LED with $V_{FD} = 3.0V$ @ $I_F = 100mA$ using PWM

- Two resistors R1 & R2 are required
- Transistor Q1
- External Power supply +Vcc

To calculate the frequency , if PWM-Voltage =1.0V , $V_{max} = 2.0V$ & T_{on}= 0.5mS



Solution:

$$PWM{-}Voltage = T_{on} \times f \times V_{max}$$

$$f = \frac{PWM - Voltage}{T_{on} \times V_{max}} = \frac{1.0}{0.5 \times 10^{-3} \times 2.0} = 1.0kHz$$

$$f = 1.0kHz$$

$$P_{loss(Q1)} = V_{CE(sat)} \times I_C \times Duty_Cycle$$

Note: The higher the PWM frequency the smaller the footprint of R1, R2 & Q1(low-power) PWM signal significantly reduces the power dissipation on the resistors and the transistor Q1

O. TAMIMI

Arduino PWM

- There are 6 PWM pins available on the Arduino-UNO board and the Pins number are 3, 5, 6, 9, 10 &11
- The analogWrite( ) command is used to address the PWM pins
- The PWM pins are 8-bit, terming that you can set the duty cycle between 0 -255

analogWrite(pin, duty-cycle)

Parameters *pin:* Arduino PWM pin (integer) or a variable (global-variable)

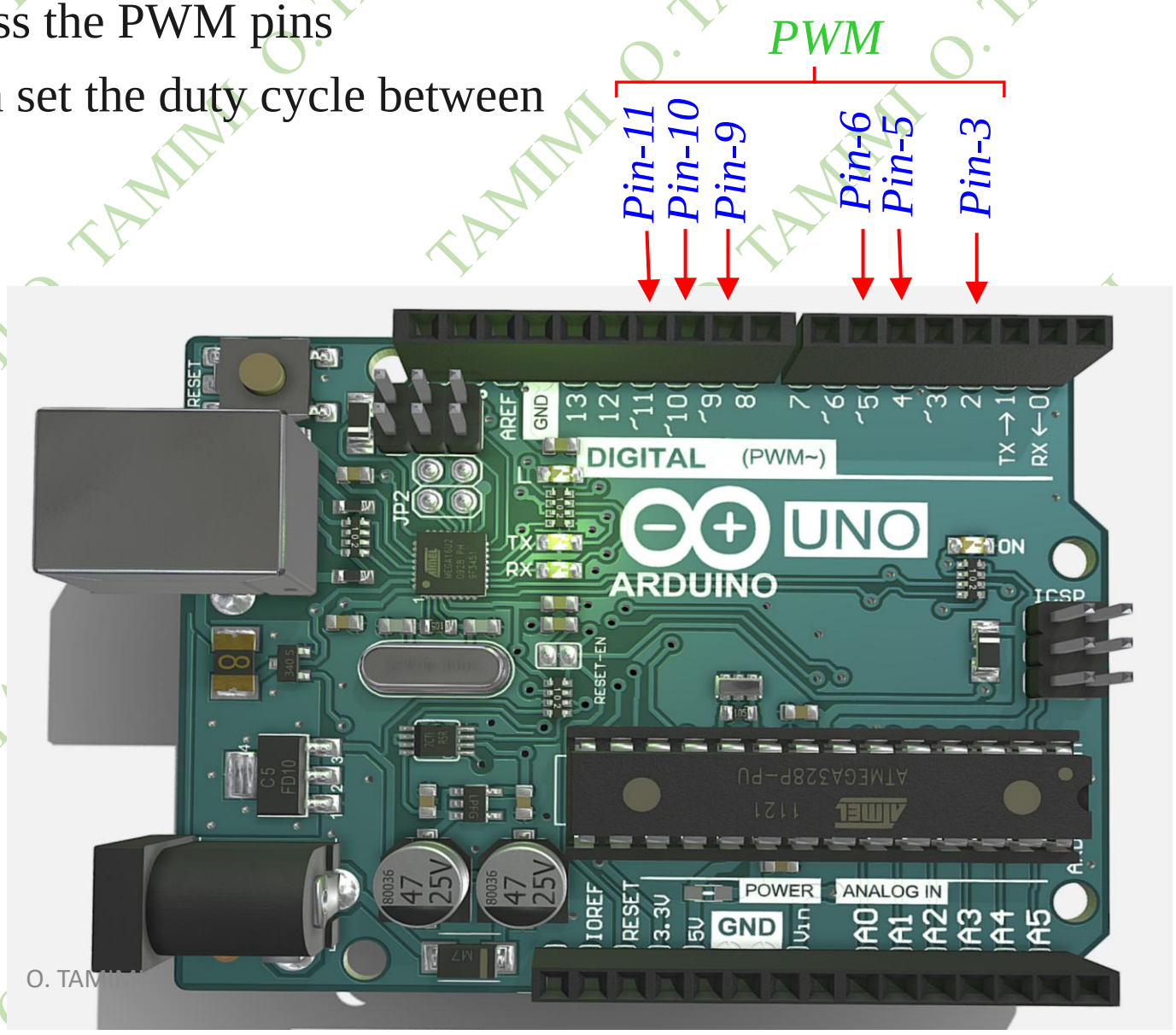
duty-cycle: Between 0-255

analogWrite(pin, duty-cycle, frequency)

Parameters *pin:* Arduino PWM pin (integer) or a variable (global-variable)

duty-cycle: Between 0-255

frequency: PWM frequency



Duty-Cycle (Digital-Value vs Output-Voltage)

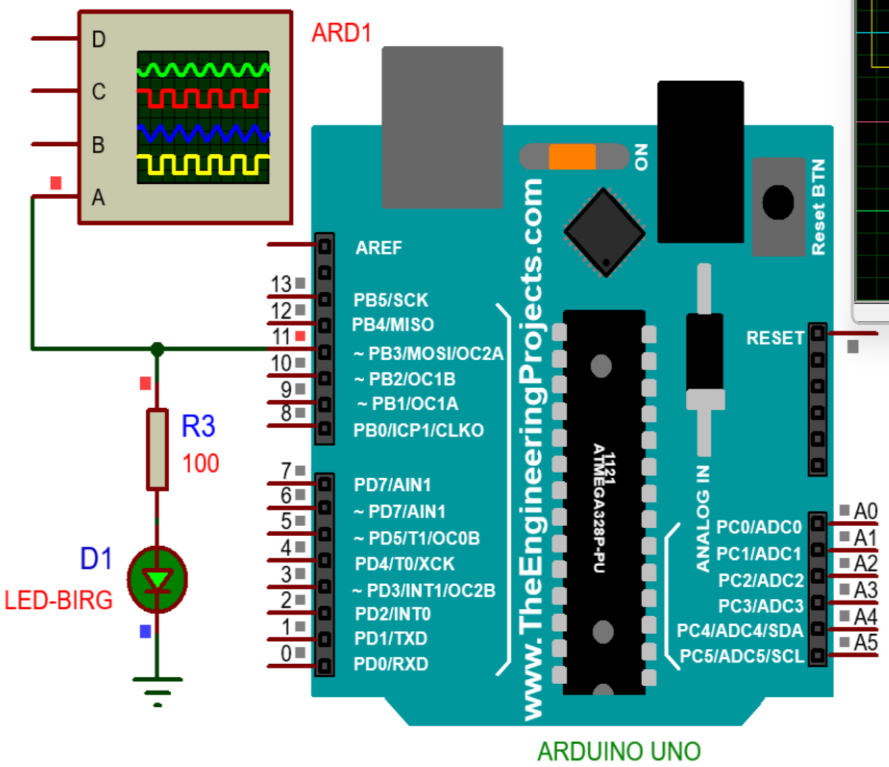
- The second argument in analogWrite( ) function is a duty cycle. The duty cycle can vary between 0 to 255 which maps 0 to 100% duty cycle in percentage
- That means we can have 256 different voltages from 0V to 5V
- The difference between each step would be 5V/255 = 0.0196V
- The analogWrite( ) function can generate PWM with a default frequency, which depends upon the pin number

PWM-Function AnalogWrite(pin, duty-cycle)	Duty-Cycle %	Average-Voltage
AnalogWrite(10,255)	100%	$V_{out} = V_{CC} = 5.0V$
AnalogWrite(10,127)	50%	$V_{out} = 2.5V$
AnalogWrite(10,0)	0%	$V_{out} = 0V$

PWM-Function AnalogWrite(pin, duty-cycle)	Fixed Frequency
Arduino-Uno Pins 3,9,10 & 11	490Hz
Arduino-Uno Pins 5 & 6	980Hz

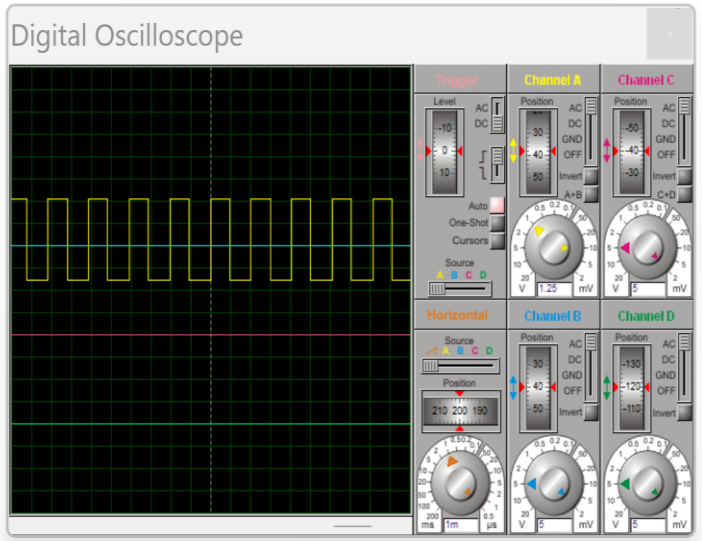
Controlling LED Brightness Using PWM

- Make sure that the connections are made as indicated in the wiring diagram
- Connect the positive leg of LED (Longer-Lead) to the digital pin-5 of Arduino
- Then connect the 220Ω resistor to the negative leg of LED and connect the other end of resistor to the ground pin of Arduino

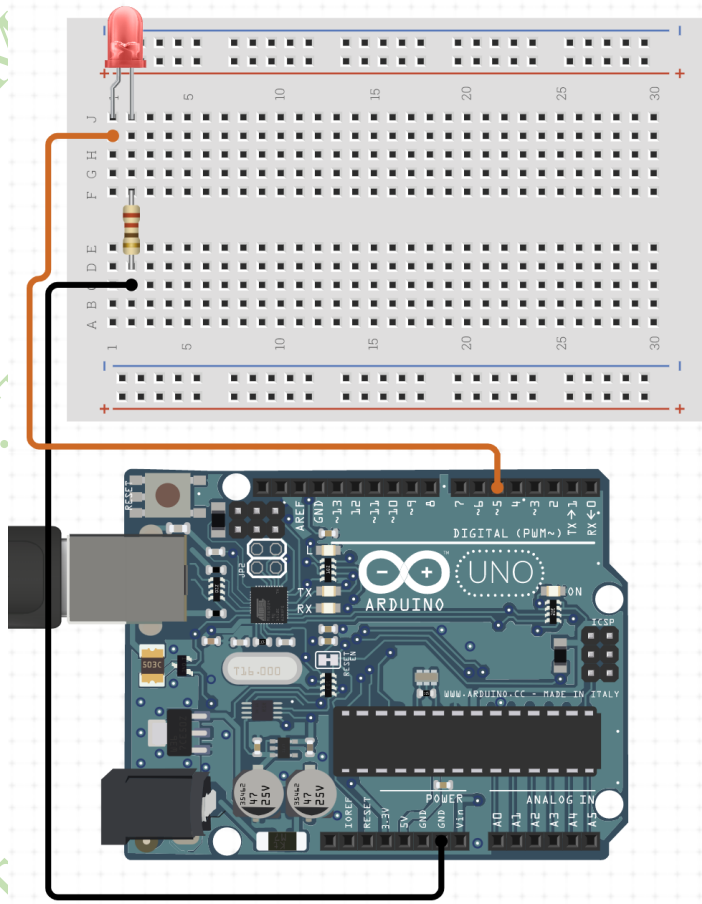


ARDUINO UNO

Arduino Schematic with Oscilloscope



Note: The Oscilloscope shows the PWM signal as a train of pulses, Ton varies with time according to the program



Arduino Wiring Diagram

```
1   const int LED1 = 5; // Connect LED1 to Arduino digital pin-5
2   int i=0;            // Initialize the duty-cycle using variable i
3
4   void setup(){       // Setup code is processed one time
5   pinMode(LED1,OUTPUT);// Asign LED1 as a digital output
6   }
7   void loop(){        // Loop function runs contineously
8   while (i<=255){     // The embedded statements inside the while
9   //                    Statement are processed as long as i<=255
10  analogWrite(LED1,i); // Initialize PWM at pin-5 with a variable i to control the
11  //                    duty-cycle, changing the duty-cycle has a direct effect
12  //                    on the LED brightness
13  delay(100);         // The duty-cycle(%) takes about 25.5 second to rise
14  //                    from 0-100% because of the time-delay
15  i++;                // Increment the value of i by one in each cycle
16  }
17  i = 255;            // Setup i at 255=2^8
18  while (i>=0){       // The embedded statements inside the 2nd while statement
19  //                    Will be proccedd as long as i>=0
20  analogWrite(LED1,i); // The duty cycle at the same pin starts to fall from
21  delay(100);         // 100% to 0% during the same interval which is 25.5 second
22  i--;                // Decrement the integer i by one in each cycle
23  }                   // Once i becomes less than 0, the process exits the while
24  }                   // to start over from beggining again
```

O. TAMIMI

Code-Analysis

- The first block of code we encounter is the declaration and initialization of two integer variables
- const int is a standard way to attach unchanged variable LED1 to digital pin-11 (Arduino-pin)
- The variable integer i will be used to specify the duty-cycle which controls the LED brightness

```
1   const int LED1 = 5; // Connect LED1 to Arduino digital pin-5
2   int i=0;            // Initialize the duty-cycle using variable i
```

- The code inside the setup( ) curly bracket will only be executed a single time by the Arduino
- The only thing we do here is set the mode of pin-5 as digital output using the pinMode( ) function

```
4   void setup(){       // Setup code is processed one time
5   pinMode(LED1,OUTPUT);// Asign LED1 as a digital output
6   }
```

- The first function we encounter in the loop( ) is while conditional statement
- The embedded statement inside the while loop will be processed if i<=255
- The analogWrite( ) function initializes the PWM capabilities to pin-5
- The variable i determines the duty-cycle of PWM signal

```
7   void loop(){          // Loop function runs contineously
8   while (i<=255){       // The embedded statements inside the while
9   //                       Statement are processed as long as i<=255
10  analogWrite(LED1,i); // Initialize PWM at pin-5 with a variable duty-cycle.
```


O. TAMIMI

- It is obvious that the duty-cycle has a direct effect on the LED brightness
- The duty-cycle takes about 25.5 second to rise from 0-100% because of the time delay
- The time-delay slows-down the glowing amount of LED

```
13   delay(100);           // The duty-cycle(%) takes about 25.5 second to rise
14   //                       from 0-100% because of the time-delay
15   i++;                  // Increment the value of i by one in each cycle
```

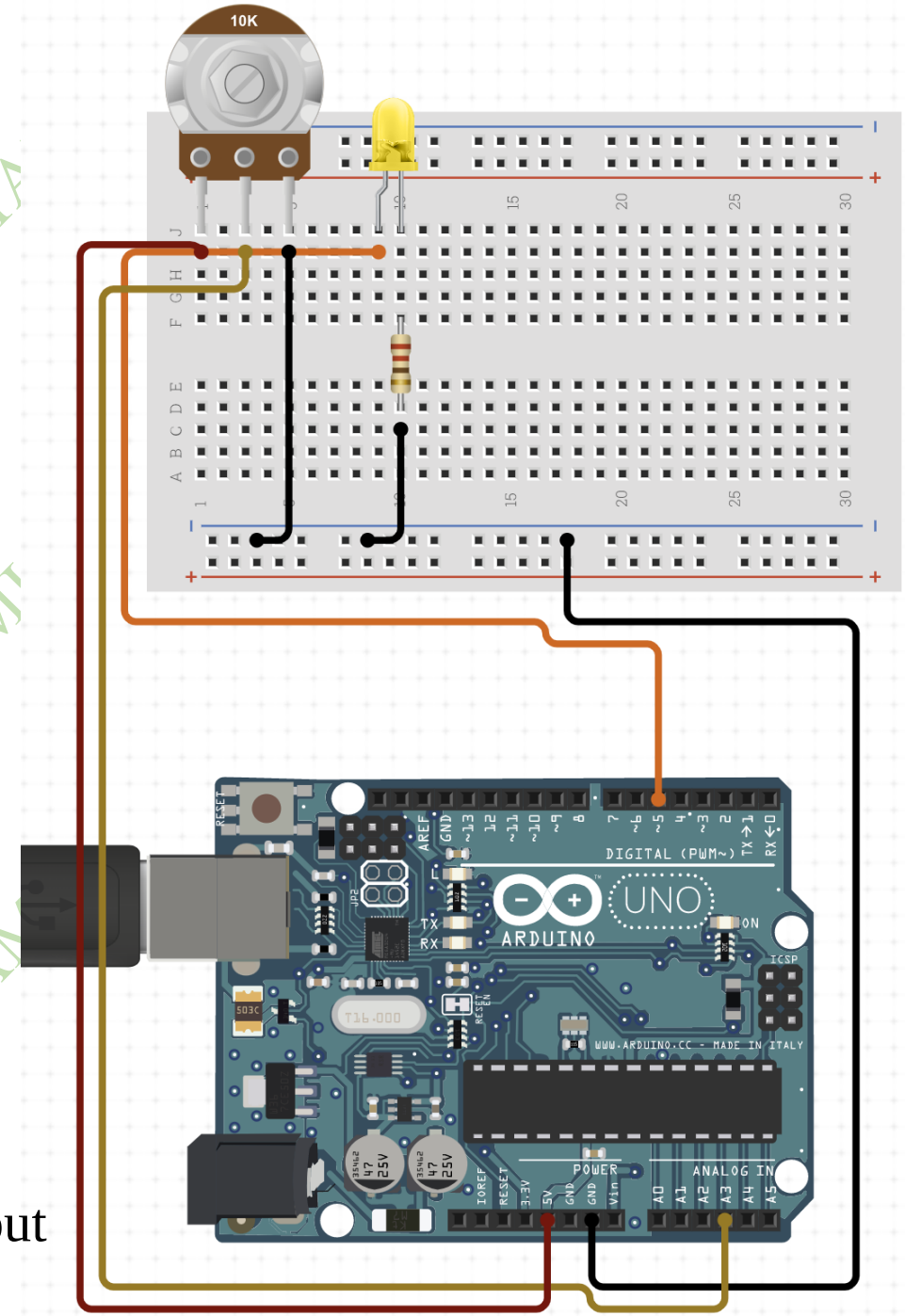
- Once i becomes greater than i>255, the program skips the while-loop to carry out the next while statement

```
18   while (i>=0){         // The embedded statements inside the 2nd while statement
19   //                       Will be proccedd as long as i>=0
20   analogWrite(LED1,i); // The duty cycle at the same pin starts to fall from
21   delay(100);           // 100% to 0% during the same interval which is 25.5 second
22   i--;                  // Decrement the integer i by one in each cycle
```

- The next while-loop, is setup with a condition (i>=0)
- The time-delay slows-down the amount of LED fading
- In each cycle inside the while-loop, the integer i will be decremented by one
- i gets decremented 255, the duty cycle falls from 100% (max LED brightness) to 0% (LED turns off)
- Once i becomes i<0, the program skips the while loop to carry on the next while-loop

Manual control the LED Brightness

- You will learn how to dynamically make the LED fade in/fade out when you turn the potentiometer knob
- Let's first build the circuit. You will need those components:
  1- Arduino board
  2- Breadboard
  3- LED – any color
  4- 220 Ohm resistor
  5- Potentiometer
  6- A bunch of wires (male to female)
- The minimum knob position will correspond to the minimum brightness – LED turned off
- And the maximum position will correspond to the maximum brightness – LED with full intensity,



Note: Connect the shorter leg of the LED to the ground via the 220 resistor

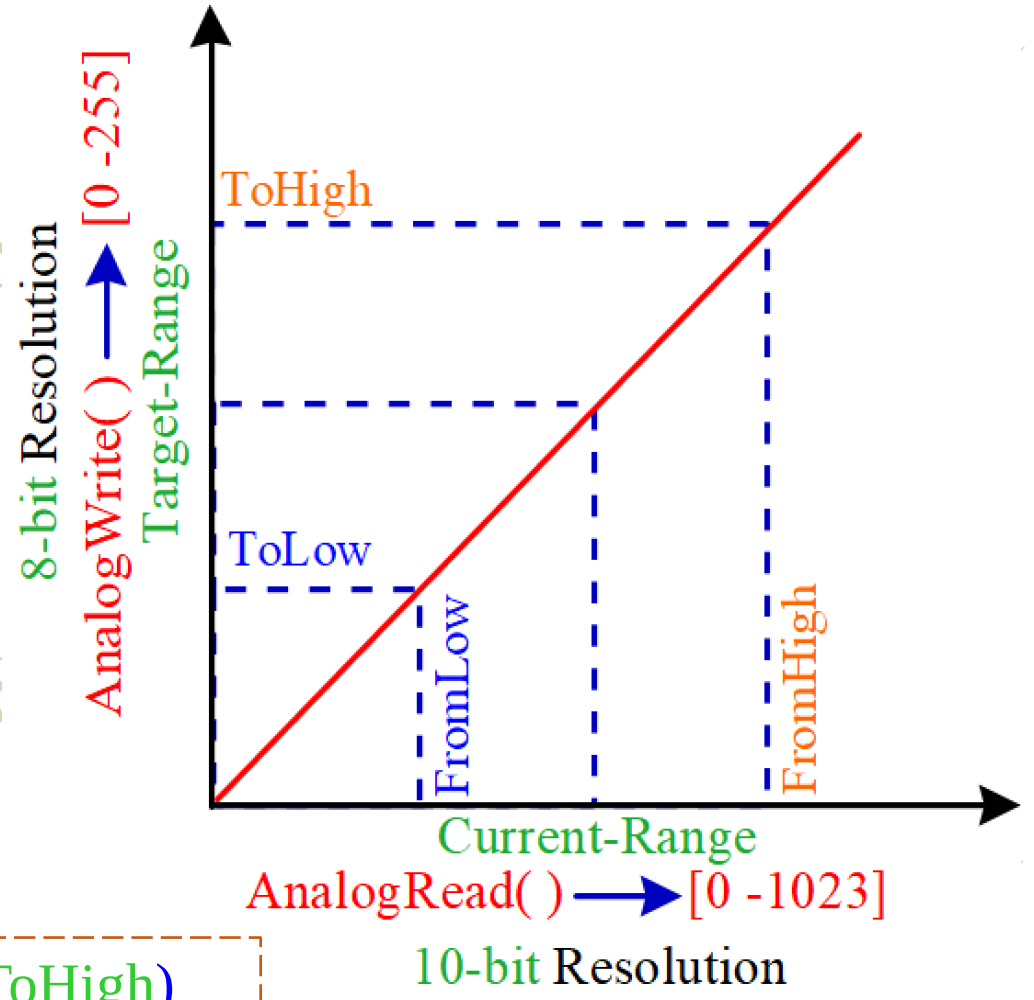
Note: The middle potentiometer pin goes directly to Analogue input pin A3

O. TAMIMI

```ino
const int LED1=6;   // Assign arduino PWM pin to LED
const int POT1=A3;  // Assign arduino analogue input A1 to potentiometer
int POT_STATE;      // Declare a variable to store potentiometer value
int BRIGHTNESS;     // Declare a variable to capture the PWM value

void setup() {
pinMode(LED1,OUTPUT); // Assign LED1 as a digital output
pinMode(POT1,INPUT);  // Assign POTENTIOMETER as an analogue input (OPTIONAL)
}                     // Closing curly bracket for the setup function

void loop() {
POT_STATE=analogRead(POT1);
//Write the analogue-pin state into a global variable POT_STATE
BRIGHTNESS=map(POT_STATE, 0, 1023, 0, 255);
//Mapping the potentiometer range 0-1023 to a new LED range 0-255
analogWrite(LED1,BRIGHTNESS);  // Apply the BRIGHTNESS to LED1
}                              // Closing curely bracket for the loop function
```



map( ) function → map(value, FromLow, FromHigh, ToLow, ToHigh)

- To remap a number from one range to another
- FromLow & FromHigh → The lower & upper range of current-value
- ToLow & ToHigh → The lower & upper range of target-value

- Here we create four integer variables to hold four different data. LED1 and POT1 hold the pin number used to connect the LED and Potentiometer to the Arduino
- POT_STATE is used to hold the input value from the potentiometer
- BRIGHTNESS is used to capture the PWM value

```
1  int LED1 = 6;   // Assign Arduino PWM pin to LED
2  int POT1 = A1;  // Assign Ardunio analogue input A1 to POTENTIOMETER
3  int POT_STATE;  // Declare a variable to store the value from the potentiometer
4  int BRIGHTNESS; // Declare a variable to capture the PWM value
```

- In the setup section, we set the LED1 as a digital output
- Setup the POT1 as an analog input

```
6  void setup() {
7  |  pinMode(LED1, OUTPUT);  // Assign LED as a digital output
8  |  pinMode(POT1,INPUT);    // Assign POTENTIOMETER as an Analogue
9  |  //                        input (Optional)
10 }                          // Closing bracket for the SETUP function
```

- Within the loop section, we use analogRead( ) function to read from the potentiometer and pass that value to POT_STATE using POT_STATE = analogRead(potPin)

```
11 void loop() {
12 POT_STATE = analogRead(POT1);
13 // Write the POT_STATE into a global variable
```


O. TAMIMI

```
14   BRIGHTNESS=map(POT_STATE, 0, 1023, 0, 255);
15   //Mapping the potentiometer range 0-1023 to a new LED range 0-255
16   analogWrite(LED1,BRIGHTNESS);  // Apply the BRIGHTNESS to LED1
17   }                              // Closing curely bracket for the loop function
```

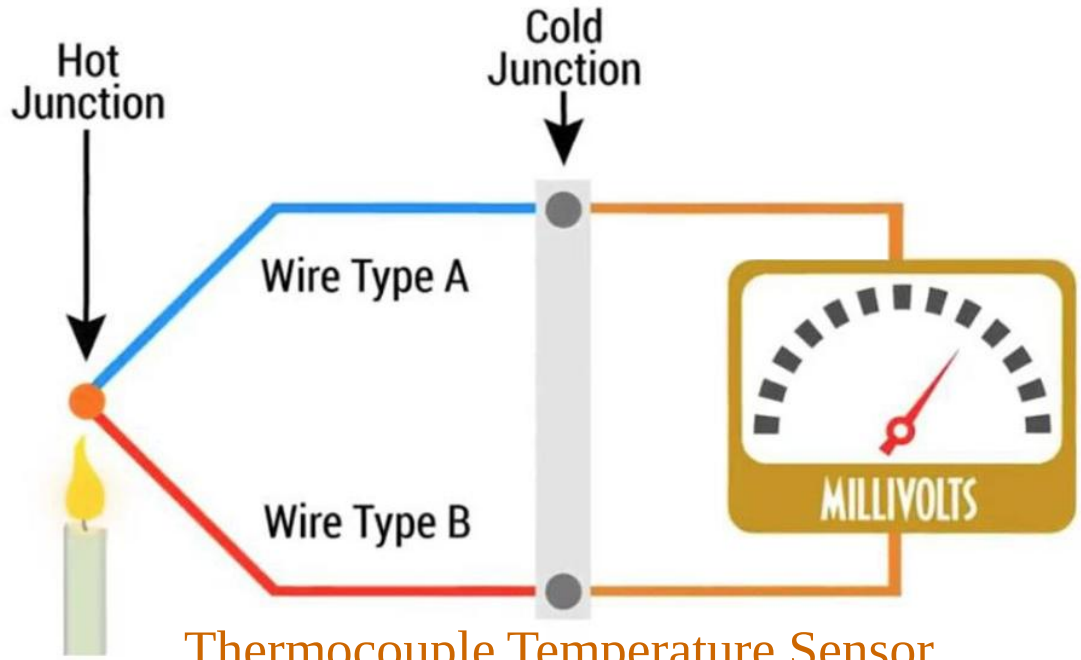
- ADC is 10-bit, so it reads a range from [0 to 1023] wile PWM is 8-bit works from [0 to 255] range
- So we have to map the current range [0 to 1023] to the current range [0 to 255]
- We can do this using the Arduino map( ) function BRIGHTNESS= map(POT1, 0, 1023, 0, 255);
- We will hold that mapping value to the variable brightness and use it within an analogWrite( ) function to change the brightness

Analogue-Devices [Temperature-Sensor]

- Temperature sensor is one of the most commonly sensor amongst other sensor types
- The temperature sensor refers to a sensor that can measure the temperature of a medium and convert it into a usable output signal
- Temperature sensors are divide into four-types, each type has its own working principle
  - Thermocouple
  - RTD (Resistance temperature detector)
  - Thermistors (NTC)
  - Semiconductor based IC

Thermocouple

- ✓ Self-powered, requires no supply
- ✓ Operate over a wide temp. range
- ✓ Has a quick response time



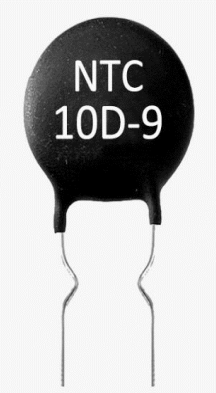
Thermocouple Temperature Sensor

Principle of operation

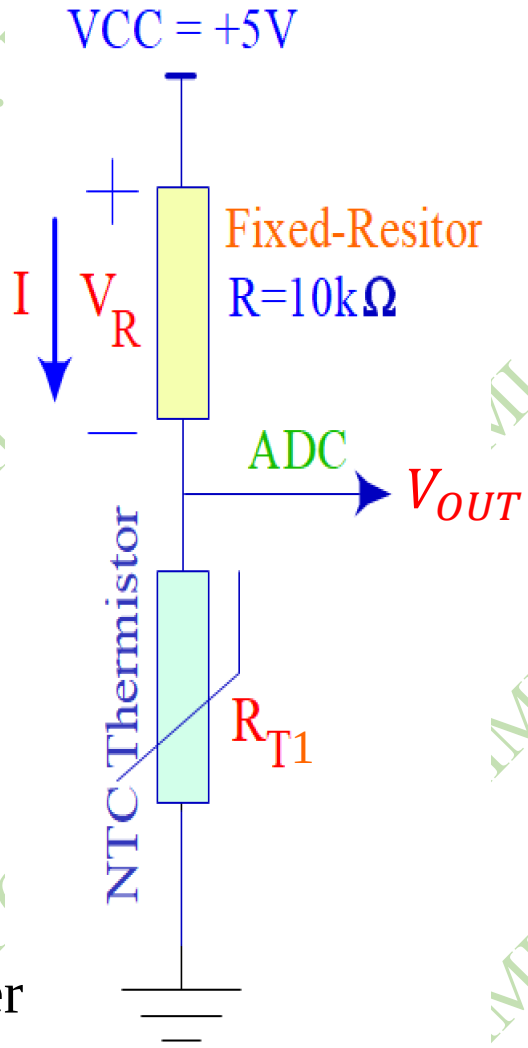
- It consists of two different types of metals, joined together at one end
- When the junction of the two metals is heated or cooled, a voltage is created that can be correlated back to the temperature

Thermistors

- ✓ Thermistors are variable resistors, whose resistance will be changed with the temperature
- ✓ The change in resistance can be correlated into a change in temperature
- ✓ Thermistors are divided into 1- NTC → Negative temperature coefficient
  2- PTC → Positive temperature coefficient



NTC Thermistor



- ✓ NTC thermistor's resistance decreases as the temperature increases
- ✓ Thermistors have a non-linear temperature resistance relationship
- ✓ To interpret the data correctly, a fixed resistor is added to form a voltage divider
- ✓ Thermistor output is digitized using ADC

$$V_{OUT} = V_{RT1} = \frac{V_{CC} \times R_{T1}}{R + R_{T1}}$$

NTC Thermistor Calculations

- To calculate the NTC thermistor resistance R_T using a simple formula called equation with parameter B

$$T_1 = \frac{1}{\frac{LN1}{B} + \frac{1}{T_0}}$$

$$LN1 = \ln\left[\frac{R_{T1}}{R_{T0}}\right]$$

R_{T1} → Final Measured Thermistor Resistance

R_{T0} → Initial Thermistor Rated Resistance at Room Temp

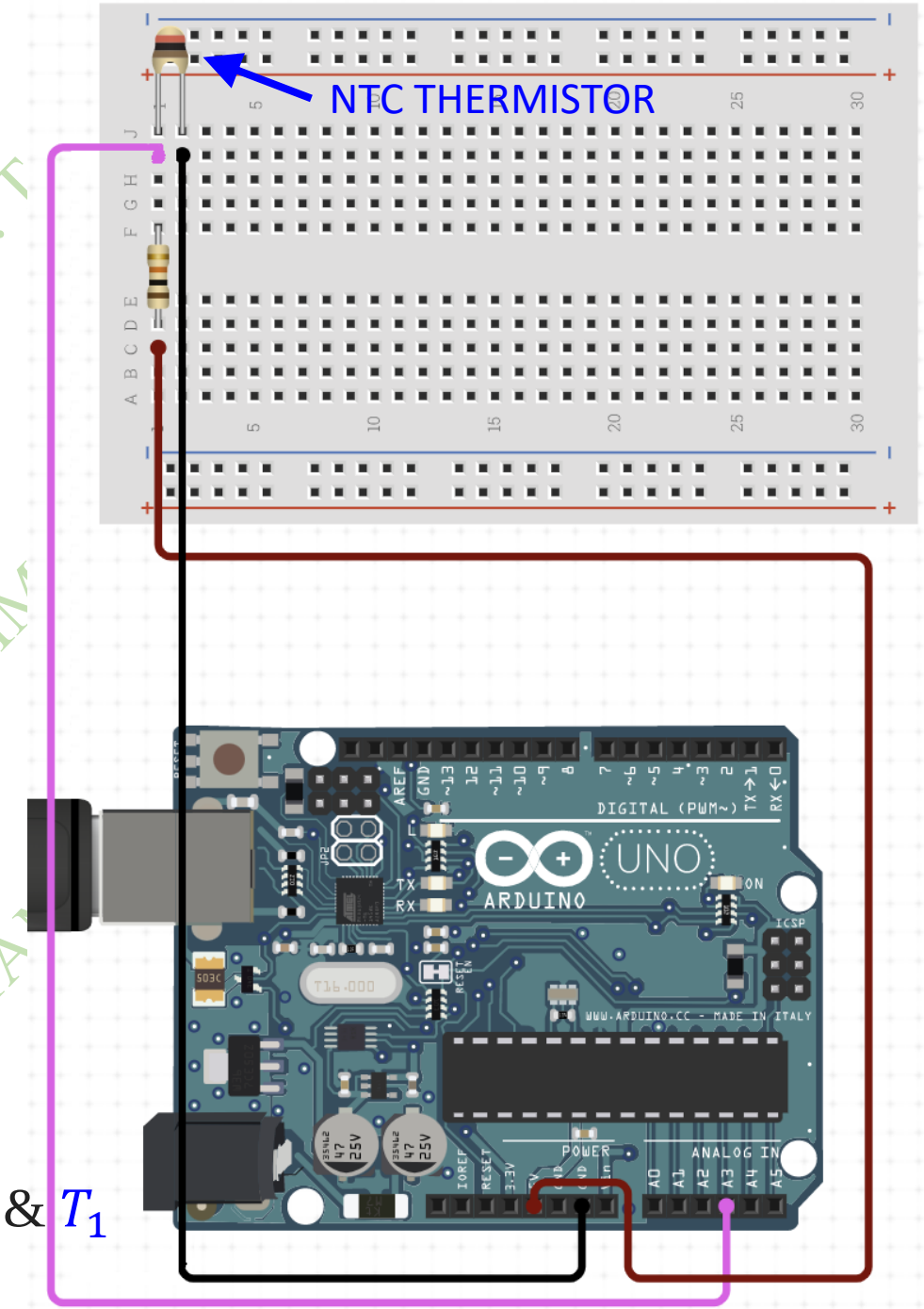
T_0 → Initial Temp. in Kelvin

T_1 → Final Measured NTC Temperature in Kelvin

$$R_{T1} = \frac{V_{RT1}}{I} = \frac{V_{RT1}}{[\frac{V_R}{R}]} = \frac{R \times V_{RT1}}{V_{CC} - V_{RT1}}$$

$[V_{RT1} = V_{out}]$

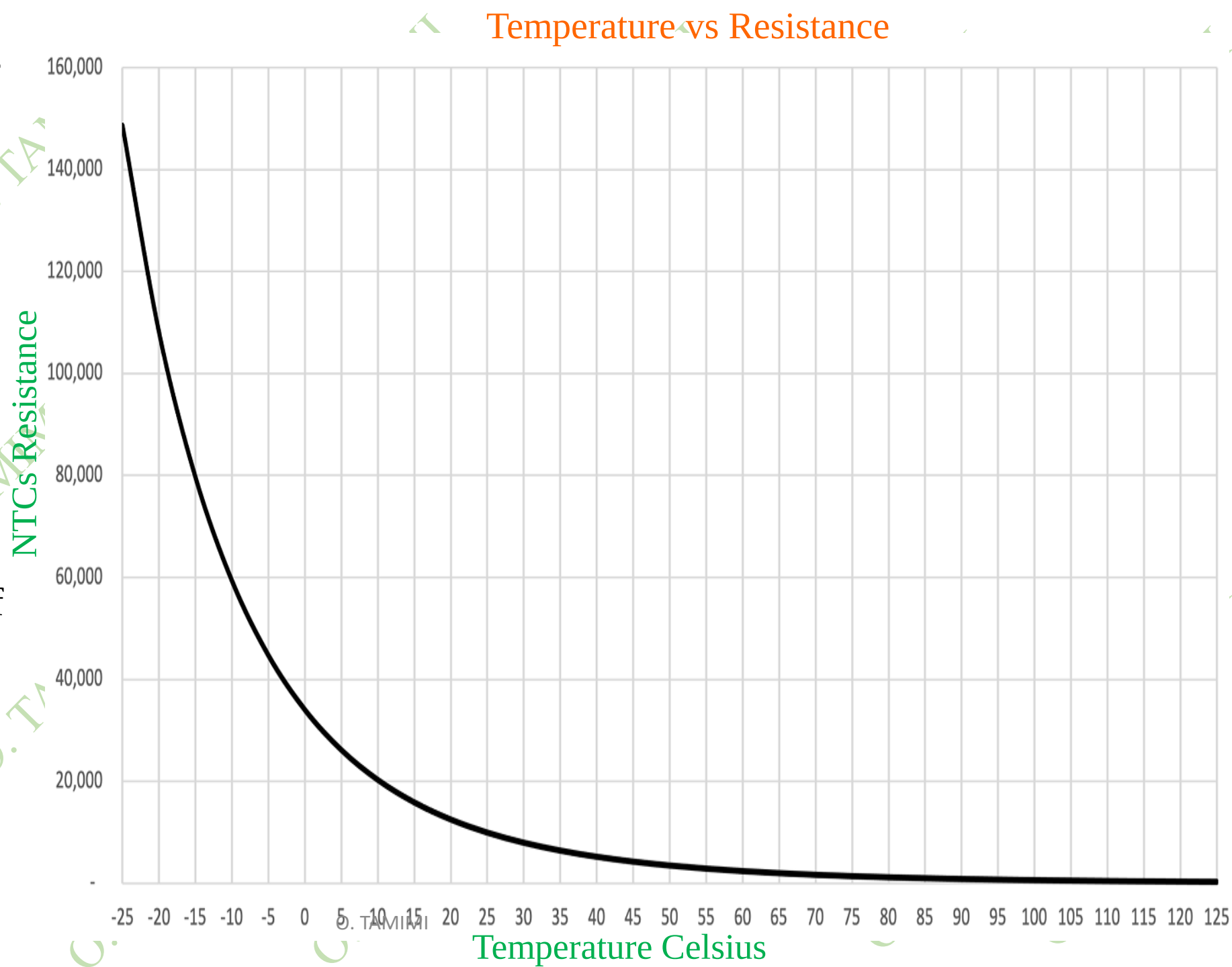
$V_{CC} = 5.0\text{V}$



Note: NTC resistance R_T is calculated using only two temp. T_0 & T_1

O. TAMIMI

- The parapolice curve shows the invers relationship between NTC's resistance and temp.
- NTCs have their resistance decreases with temp. rise
- The biggest disadvantage of thermistors is that their temperature-resistance characteristic is not linear



O. TAMIMI

Example: Write an Arduino sketch to measure the Temp. in Kelvin, Celsius, Fahrenheit using NTC Thermistor. If the NTC resistance is based temperature $R_{T0} = 10k\Omega$ $@T_0 = 25°C$. The pullup resistor value $R = 10k\Omega$

Note By referring to the previous Arduino wiring-diagram. The NTC common pin is connected to analog input pin A3

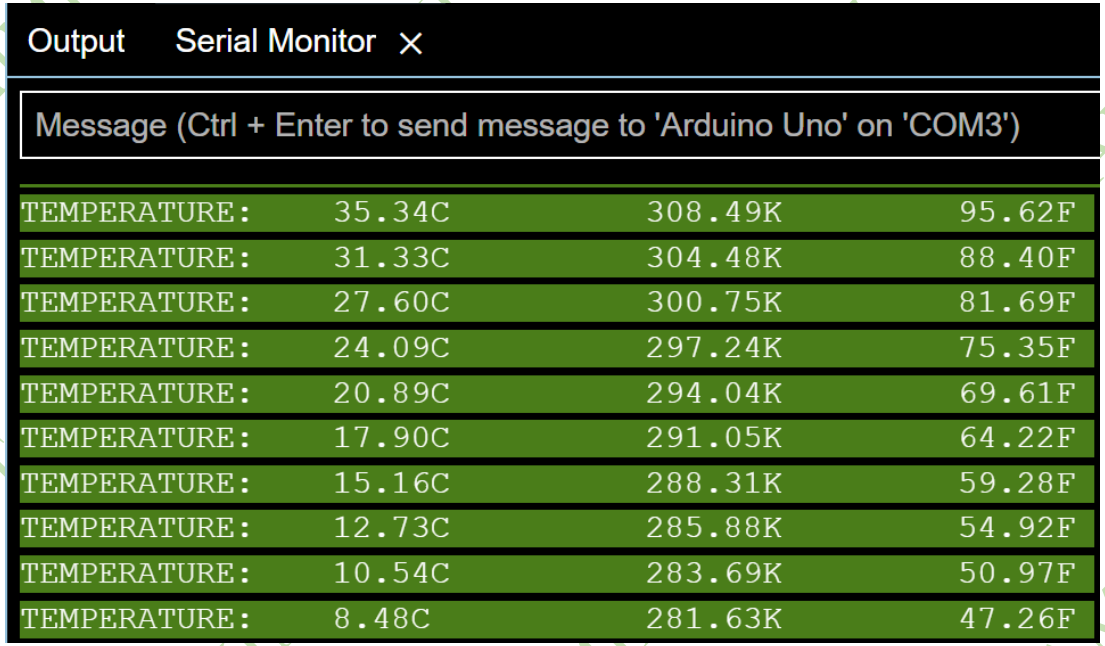
NTC pin → A3

```
/* NTC Thermistor Parameters:
 * RT0: 10K-Ohm Thermistor Resistance @ Room-Temp 25C
 * B: 3977K +- 0.75%
 * T0: 25C Initial Temp.
 * #define is a Preprocessor Directive Which Defines Constants Within The Code
*/
#define RT0 10000 // Thermistor-Resistance @ Room-Temp.
#define B   3977  // Thermistor Constanct
#define VCC 5     // Supply Voltage VCC=+5.0V
#define R 10000   // Fixed Resistor Value R=10K
float RT1, VRT1, LN1, T0, T1; // RT: Measured NTC Temp., VRT1: ADC-Voltage
                              // LN1: Logarithm Equation, T0: Initial-Temp. T1: Final Calculated Temp.
int ANL=A3;

void  setup(){   // Setup Subroutine Executed Once
Serial.begin(9600); // Initialize the UART @ 9600 BPS
T0=25+273.15;       // Convert T0 from Celsius to Kelvin
}
void loop(){                  // Starting of the Loop Subroutine
VRT1=analogRead(ANL);         // Aquire ANL Value Into VRT1 Variable
VRT1= (5.00/1024)*VRT1;       // Convert VRT1 From Quantlization-Level into Voltage-Step
RT1= (VRT1*R)/ (VCC-VRT1); // Calculate the New Resistance Using Voltage-Divider Rule
LN1=log(RT1/RT0);             // Take the Logarith of New to Old NTC Resistance
T1=(1/(LN1/B) +  (1 / T0));// Calculate the New NTC Temp. T1 in Kelvin
T1=T1-273.15;                 // Convert T1 Temp. From Kelvin into Celsius
Serial.print("Temp: ");       // Print Temp String
Serial.print("\t");           // Insert Double Space
Serial.print(T1);             // Print the New NTC Temp in Celsius
Serial.print("C\t\t");        // Print the Unit C as a String
Serial.print(T1+263.15);      // Convert T1 into Kelvin and Display the Result
Serial.print("K\t\t");        // Print the Unit K as a String
Serial.print((T1*1.8)+32); // Convert T1 into Fahrenheit and Display the Result
Serial.println("F");          // Print the Unit F as a String
}
```

O.TAMIMI

- The output temperature readings can be retrieved using the serial monitor
- According to the sketch, the temperature is given in Celsius, Kelvin and Fahrenheit
- The program updates the temp. reading every half a second



Semiconductor temperature sensors

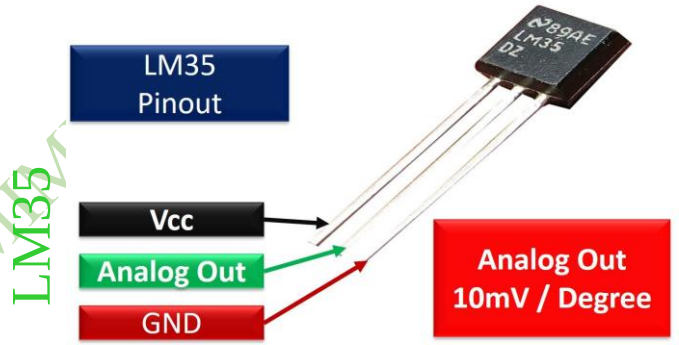
- Semiconductor temperature sensors are electronic devices which come in the form of integrated circuits
- The most popular ones include DS18B20 and the LM35
- They provide reasonably linear output
- Their temperature range is typically limited between -40 to +120°C
- Semiconductor temperature sensors are divided into

1) Analogue output temperature sensor

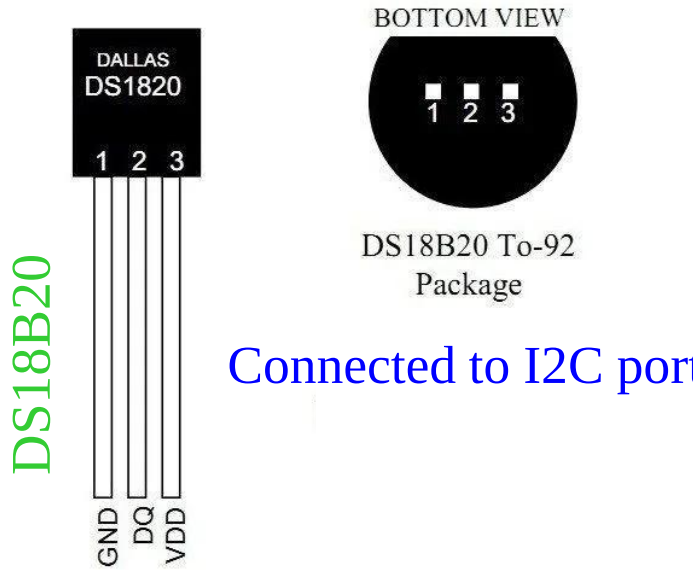
These type generates a voltage or current output signal which is linear and directly proportional to temperature

2) Digital output temperature sensor

Digital temp. sensors provide a serial digital signal as indicated in the following forms I2C or SPI



Analog-Output Temp. Sensor

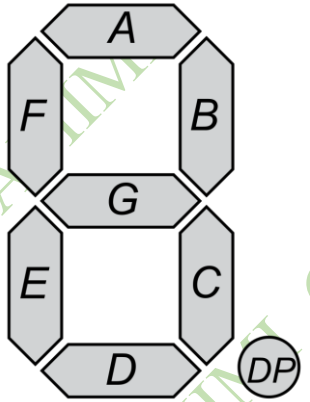


Connected to I2C port

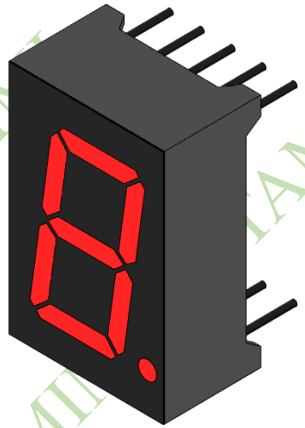
Digital-Output Temp. Sensor

Seven-Segment Display

- Seven-segment displays uses a light emitting diode (LED) for each segment
- The seven segments are arranged as a rectangle of two vertical segments on each side with one horizontal segment on the top, middle, and botto
- The seven elements of the display can be lit in different combinations to represent the nine numerical digits 0,1,2,3,4,5,6,7,8 & 9
- The segments are referred to by the letters A to G and the decimal-point
- The next lookup table may be useful for writing code to drive a 7-segment display



Segments vs Char.



7-Segment Display

Digit	Display	A	B	C	D	E	F	G
0	0	on	on	on	on	on	on	
1	1		on	on				
2	2	on	on		on	on		on
3	3	on	on	on	on			on
4	4		on	on			on	on
5	5	on		on	on		on	on
6	6	on		on	on	on	on	on
7	7	on	on	on				
8	8	on	on	on	on	on	on	on
9	9	on	on	on	on		on	on

Truth-Table for 7-Segment Display [Single-Digit]

O. TAMIMI

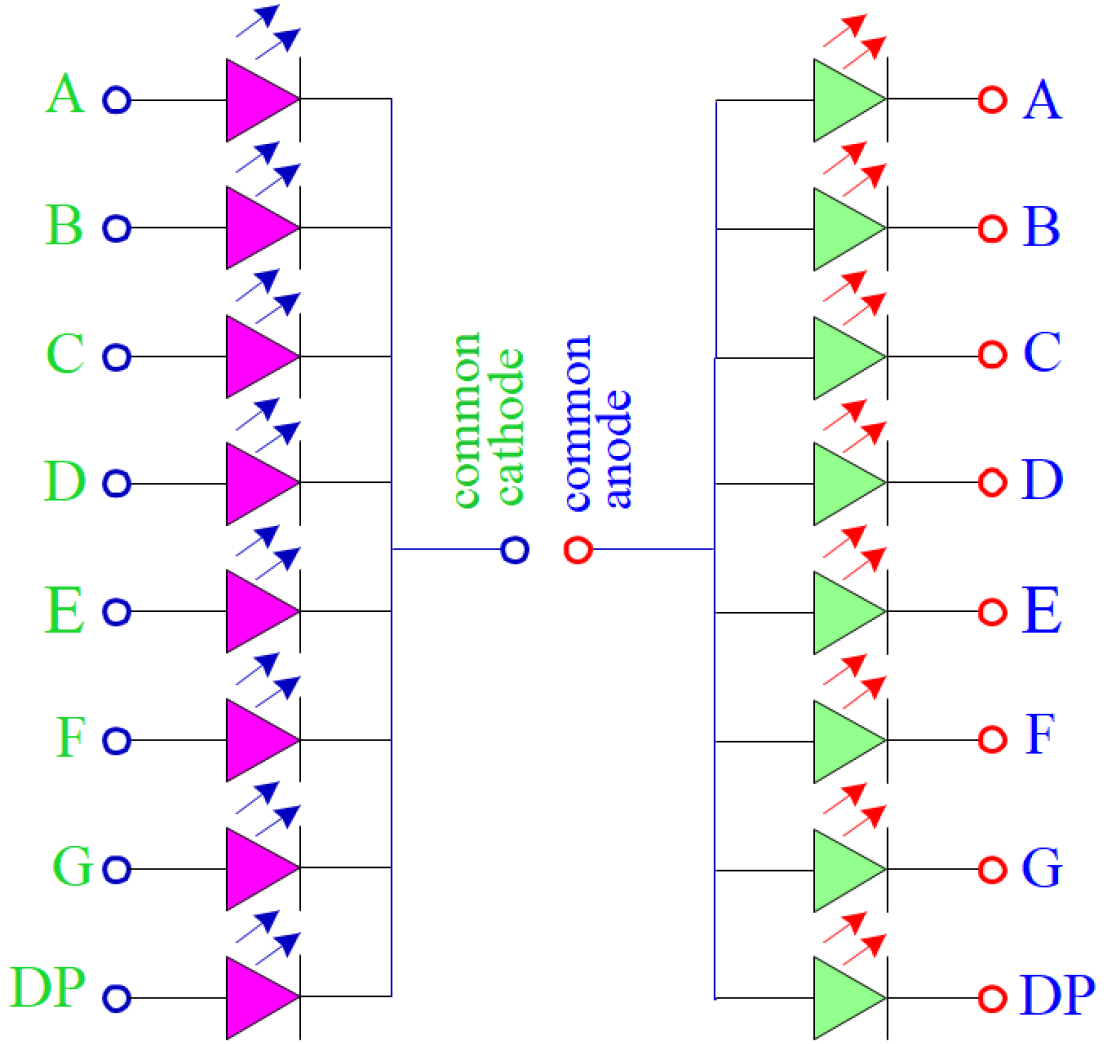
O.TAMIMI

Types of 7-Segment Display

- There are two types of LED 7-segment displays
  - 1- Common Cathode (CC)
  - 2- Common Anode (CA)
- In a Common Cathode type, the cathode of all the 8 LEDs are connected together and made common for the entire display unit
- In a Common Anode type, the anodes of all the 8 LEDs are connected together and made common for the entire display unit



7-Segment Display

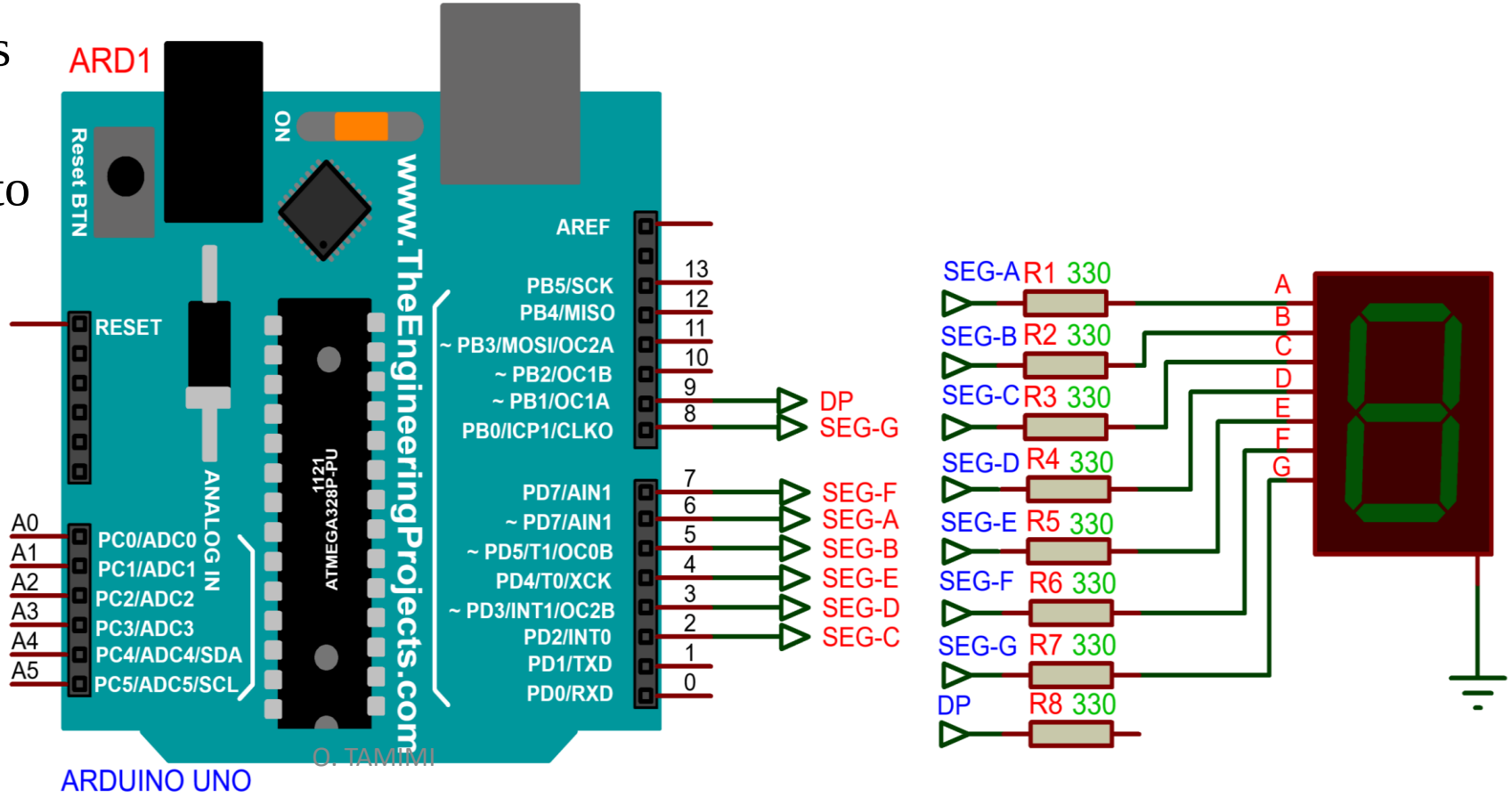


LEDs configuration inside the 7-Segments

O. TAMIMI

How to Interface 7-Segment Display with Arduino-Uno

- According to the circuit, the 7-Segment display is directly driven by Arduino
- Resistors are needed between each LED segment and the Arduino-Uno
- The complexity of Arduino sketch depends upon the way we are going to display the segments (counting up of numerical digits)
- We use here a common cathode display, hence the common pin is connected to GND
- The pins have to be driven high to turn on the corresponding LED segments



O. TAMIMI

SevSeg Library

- This library turns your Arduino into a seven segment display controller
- Use it to easily display numbers on your seven segment display without any additional controllers
- It supports common cathode and common anode displays
- This library can be used to display decimal or hexadecimal numbers and characters
- In the following 7-segment programs, the segment pins connection to the Arduino according to this table

Segment Pin	Arduino Pin
A	6
B	5
C	2
D	3
E	4
F	7
G	8
DP	9

7-Segment Configuration

#include “SevSeg.h”	Initialize the Seven Segment Library
SevSeg sevseg	Create SevSeg Object
byte numDigits = 1	To Setup the Number of Digits on the Seven Segment
byte digitPins[ ] = { }	Creates an array that defines the ground pins when using a 4 digit display. Leave it empty if you have a single digit display
byte segmentPins[ ] = {6, 5, 2, 3, 4, 7, 8, 9}	This Declares an Array that Defines which Arduino Pins are Connected to Each Segment of the Display. The Order is Alphabetical (A, B, C, D, E, F, G, DP)
bool resistorOnSegment=true	Set to True if a Current Limiting Resistors are Inserted in Series with the Seven Segment Display
byte hardwareConfig =COMMON_CATHODE	To Configure the Type of Seven Segment Display
sevseg.begin(hardwareConfig, numDigits, digitPins, segmentPins, resistorsOnSegments)	This Object is part of the initialization process for a seven-segment display, which includes multiple parameters
sevseg.setBrightness(90)	To Adjust the Brightness of the Display
sevseg.setNumber( )	This Function Prints The Number To The Display
sevseg.refreshDisplay( )	This function is required at the end of the loop section to continue displaying the number

Example: Write an Arduino sketch to display no.4 on a single-digit common-cathode 7-segment display
The sketch uses a SevSeg.h library (which is compatible with Arduino-IDE)
Note: The display has 8-pins, 7-pins are connected to 7-LED segments & the last one is common to ground

```
#include "SevSeg.h" //Use the SevSeg library
SevSeg sevseg;

void setup(){ //Setup code goes here
    byte numDigits = 1; // We are using a single digit display
    byte digitPins[] = {}; // We often leave it empty for single-digit display
    byte segmentPins[] = {6, 5, 2, 3, 4, 7, 8, 9}; // This declares an array that defines which
    //                                                Arduino pins are connected to each segment
    bool resistorsOnSegments = true; // This needs to be set to true if your current limiting
//                                     resistors are in series with the segment pins

    byte hardwareConfig = COMMON_CATHODE; //Set the type of 7-segment--> (Common-Cathode)
    sevseg.begin(hardwareConfig, numDigits, digitPins, segmentPins, resistorsOnSegments);
    sevseg.setBrightness(90); // To adjust the brightness of 7-segment display
}
void loop(){
sevseg.setNumber(4);// This function prints No.4 to the display
sevseg.refreshDisplay();// This function is required to continue displaying the number
}
```

Page-183

Example: Use a single common-cathode 7-segment display to count up from 0-9. the display is connected to Arduino-Uno module using a set of current limiting resistors 330Ω

Page-184

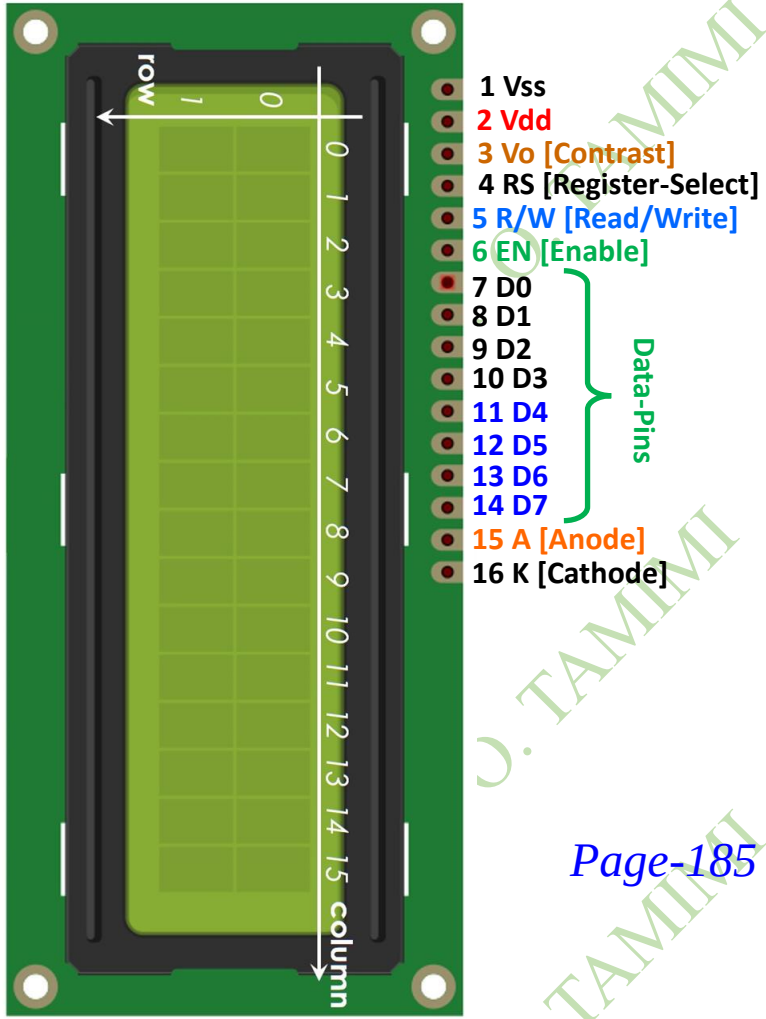
```
#include "SevSeg.h" //Use the SevSeg library
SevSeg sevseg;

void setup(){ //Setup code goes here
byte numDigits = 1; // We are using a single digit display
byte digitPins[] = {}; // We often leave it empty for single-digit display
byte segmentPins[] = {6, 5, 2, 3, 4, 7, 8, 9}; // This declares an array that defines which
//                                              Arduino pins are connected to each segment
bool resistorsOnSegments = true; // This needs to be set to true if your current limiting
//                                          resistors are in series with the segment pins

byte hardwareConfig = COMMON_CATHODE; //Set the type of 7-segment--> (Common-Cathode)
sevseg.begin(hardwareConfig, numDigits, digitPins, segmentPins, resistorsOnSegments);
sevseg.setBrightness(90);    // To adjust the brightness of 7-segment display
}
void loop(){
for(int i = 0; i < 10; i++){ // The for-loop counts up frmo i=0 to i=9
sevseg.setNumber(i);         // This function display i
delay(1000);                 // Pause for 1 second time
sevseg.refreshDisplay();     // This function is required to continue displaying the number
}
}
```

2x16 LCD Display

- The term LCD stands for liquid crystal display. It is one kind of electronic display module used to display messages and data
- 16×2 LCD As the name suggests, it includes 16-Columns & 2-Rows so it can display 32-characters (16×2=32) in total
- Every character will be made with 5×8 (40) Pixel Dots
- So the total pixels within this LCD can be calculated as 32 x 40=1280



2x16 LCD Display

LCD Pin-Diagram

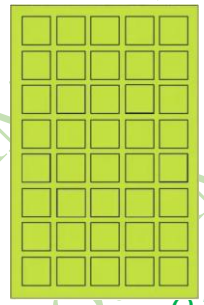
Pin-1 → Vss=GND & Pin-2→Vdd=+5.0V

Pin-3 → To control the brightness, max brightness → if Pin-3 =VCC

Pin-4→ Register-Select (RS) to receive either 0 or 1 from the Arduino-Uno
Logic-state Low 0→ Data-Mode [To send data to the display]
Logic-state High 1→ Command-Mode [Set the cursor to a specific location or delete the display]

Pin-5→ Select between Read-Mode →1 [Read the data stored in register]
Write-Mode →0 [Tied to ground to receive data]

Pin-6→ if [EN=1] enables the LCD. If [EN=0] disable the LCD



One Character [40-Pixels]

O. TAMIMI

Pin-7 D0 to Pin-14 D7 → Data-pins to receive data or commands from the Arduino-Uno
Data pins has to operation-modes: 4-bit mode and 8-bit mode
4-bit Mode: 8-bit of data can be transmitted twice (Slower). The required pins [D4-D7]
8-bit Mode: 8-bit of data can be transmitted once (Faster). The required pins [D0-D7]

Pin-15 A & Pin-16 K→ To power the backlight of LCD display
Pin-15 [Anode] connected to VCC=+5.0V DC
Pin-16 [Cathode] connected to GND

- Note: 4-bit mode is widely used because it occupies less pins on the Arduino-Uno

Interfacing the LCD display to Arduino-Uno

- To interface LCD to Arduino-Uno, the following items are requires

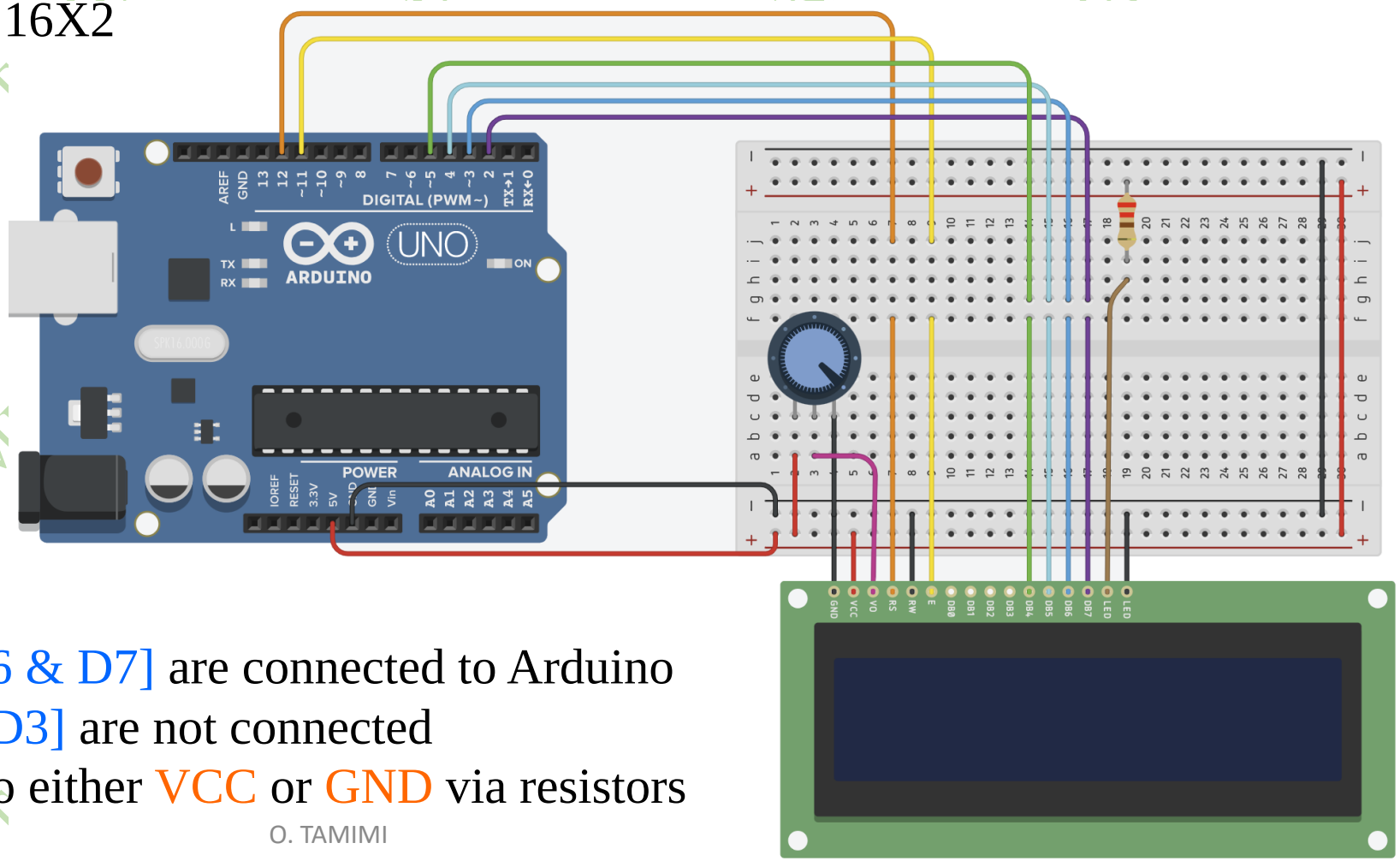
1-Arduino-Uno

2-White Bread-Board

3-Potentiometer to control the LCD brightness

4-Alphaneumerical LCD display 16X2

5-Bunch of wires



Note:

- Six pins on LCD [RS,E,D4,D5,D6 & D7] are connected to Arduino
- Four pins on LCD [D0,D1,D2, & D3] are not connected
- The remining pins are connected to either VCC or GND via resistors

How to program LCD Display

```
#include <LiquidCrystal.h>
```

- This library allow the communication with Liquid crystal display
- The library works with in either 4 or 8 bit mode [i.e. using 4 or 8 data lines]

```
const int RS = 12, EN = 11, D4 = 5, D5 = 4, D6 = 3, D7 = 2;
```

- unchanged variables have been declared here as integers to assign the LCD pins to Arduino's digital pins

```
LiquidCrystal lcd(RS, EN, D4, D5, D6, D7)
```

- This defines which pins of the Arduino to be connected to which pins of the LCD display
- The arguments for the LCD initialization represent the Arduino pins connected to RS, EN, D4, D5, D6, and D7, in that order

```
lcd.begin(16, 2);
```

- Initializes the interface to the LCD screen, and specifies the dimensions (width and height) of the display

```
lcd.setCursor(column_index, row_index);
```

- Move the cursor to the desired position

```
lcd.print("Hello World!");
```

- Print a message to the LCD like printing `Hello World!`

Example: Write an Arduino Sketch to display two simple messages on the LCD display using 4-bit mode with data pins D4,D5, D6 & D7. A Potentiometer is required to control the LCD brightness (Note: See the previous wiring-diagram and the next schematic)

```
#include <LiquidCrystal.h> //Use a special library to initialize the LCD display
const int RS = 12, EN = 11, D4 = 5, D5 = 4,D6 = 3, D7 = 2;
//Declare integer varialbes to assign LCD pins to Arduino pins
LiquidCrystal lcd(RS, EN, D4, D5, D6, D7);
// Defines the 4-Mode data transfer using D4,D5,D6 & D7
void setup() {
  lcd.begin(16, 2);              // Set up number of columns and rows
  lcd.setCursor(0, 0);           // Move cursor to (0, 0)
  lcd.print("ARDUINO");          // Print ARDUINO message at (0, 0)
  lcd.setCursor(2, 1);           // Move cursor to (2, 1)
  lcd.print("HELLO WORLD");      // Print HELLO-WORLD message at (2, 1)
}
void loop() {}
```

Code-Analysis

- The sketch demonstrates how to initialize the display, how to write text, and how to move the cursor

- Declare a constant variables as integers to assign LCD pins to the Arduino-Uno digital pins

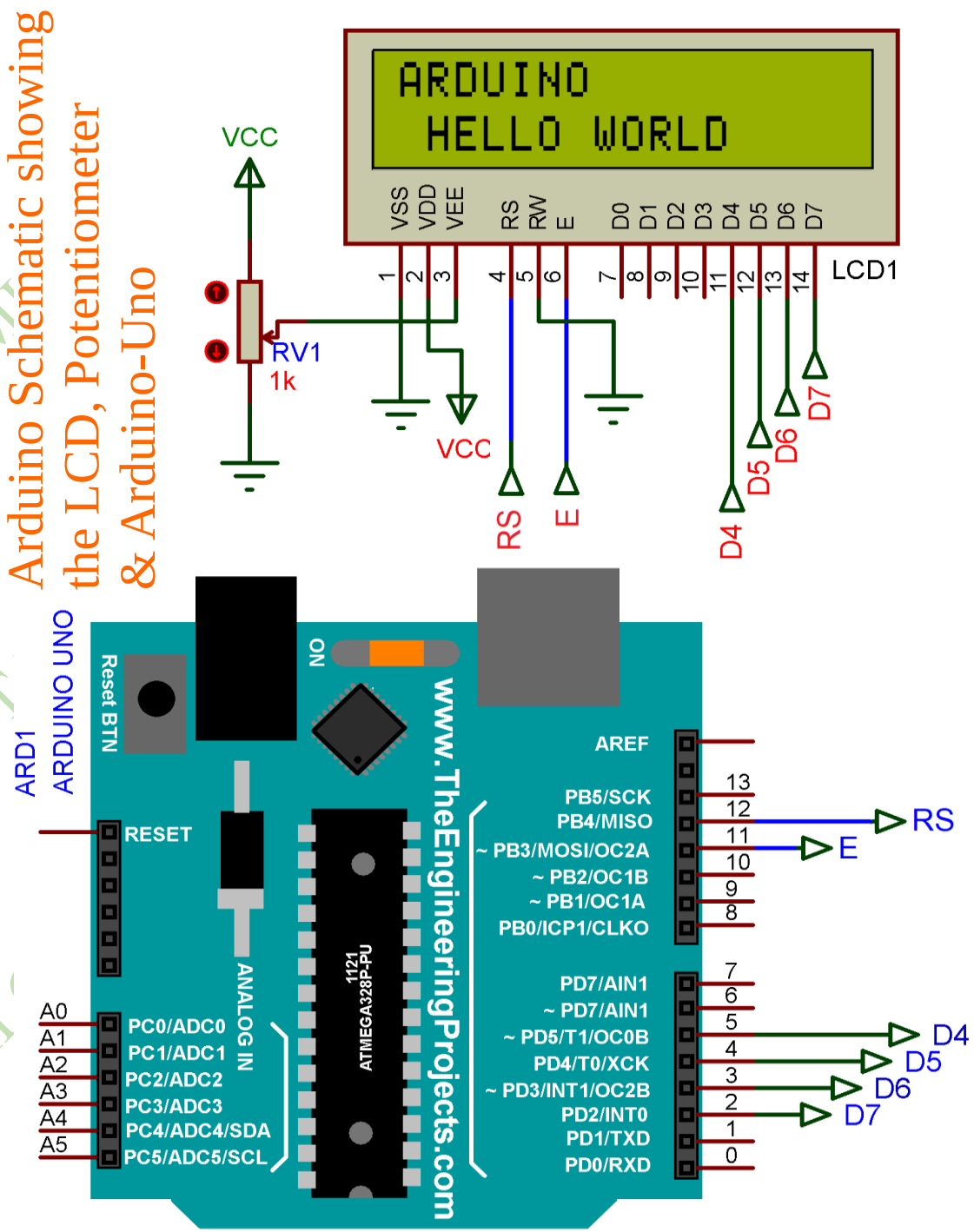
```
3   const int RS = 12, EN = 11, D4 = 5, D5 = 4,
4   D6 = 3, D7 = 2; //Declare integer varialbes
5   // to assign LCD pins to Arduino pins
```

- The arguments for the LCD initialization represent connection of Arduino digital pins to LCD pins RS, EN, D4, D5, D6, and D7, in that order

```
4   LiquidCrystal lcd(RS, EN, D4, D5, D6, D7);
```

- you call the library's `begin( )` function to set up the LCD display with the character size [16x2]

```
7     lcd.begin(16, 2);
```



O. TAMIMI

- After doing that, you can call the functions `print( )` and `setCursor( )` commands to print text to a given location on the display

```
 8   lcd.setCursor(0, 0);          // Move cursor to (0, 0)
 9   lcd.print("ARDUINO");         // Print ARDUINO message at (0, 0)
10   lcd.setCursor(2, 1);          // Move cursor to (2, 1)
11   lcd.print("HELLO WORLD");     // Print HELLO-WORLD message at (2, 1)
```

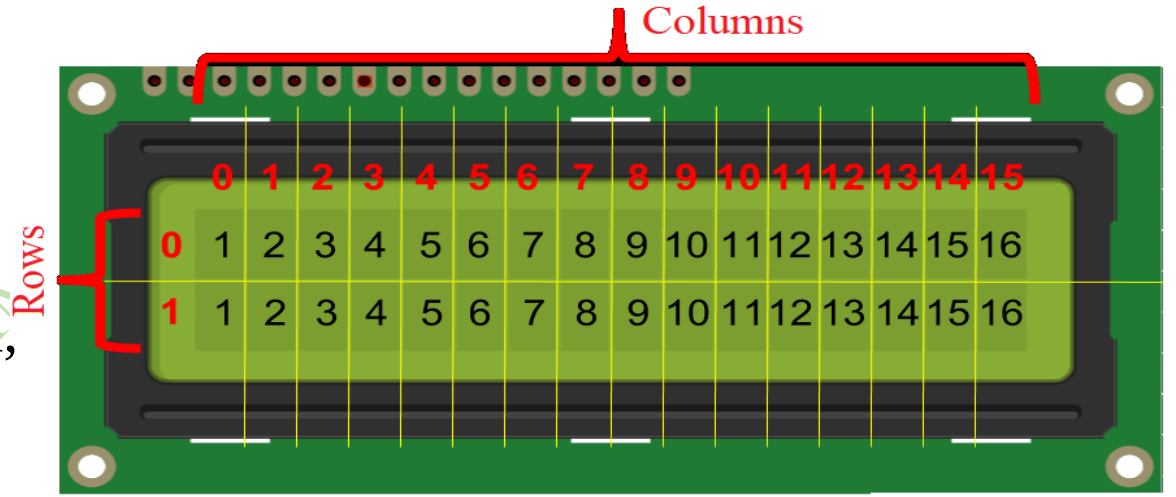
- For example, if you want to print Arduino on the second row & 1st column , you issue these commands:

```
lcd.setCursor(0, 1);
lcd.print("Arduino");
```

- The positions on the screen are indexed starting with `(0,0)` in the top-right position.

- The argument `setCursor(C,R)` specifies the column number, and the second specifies the row number

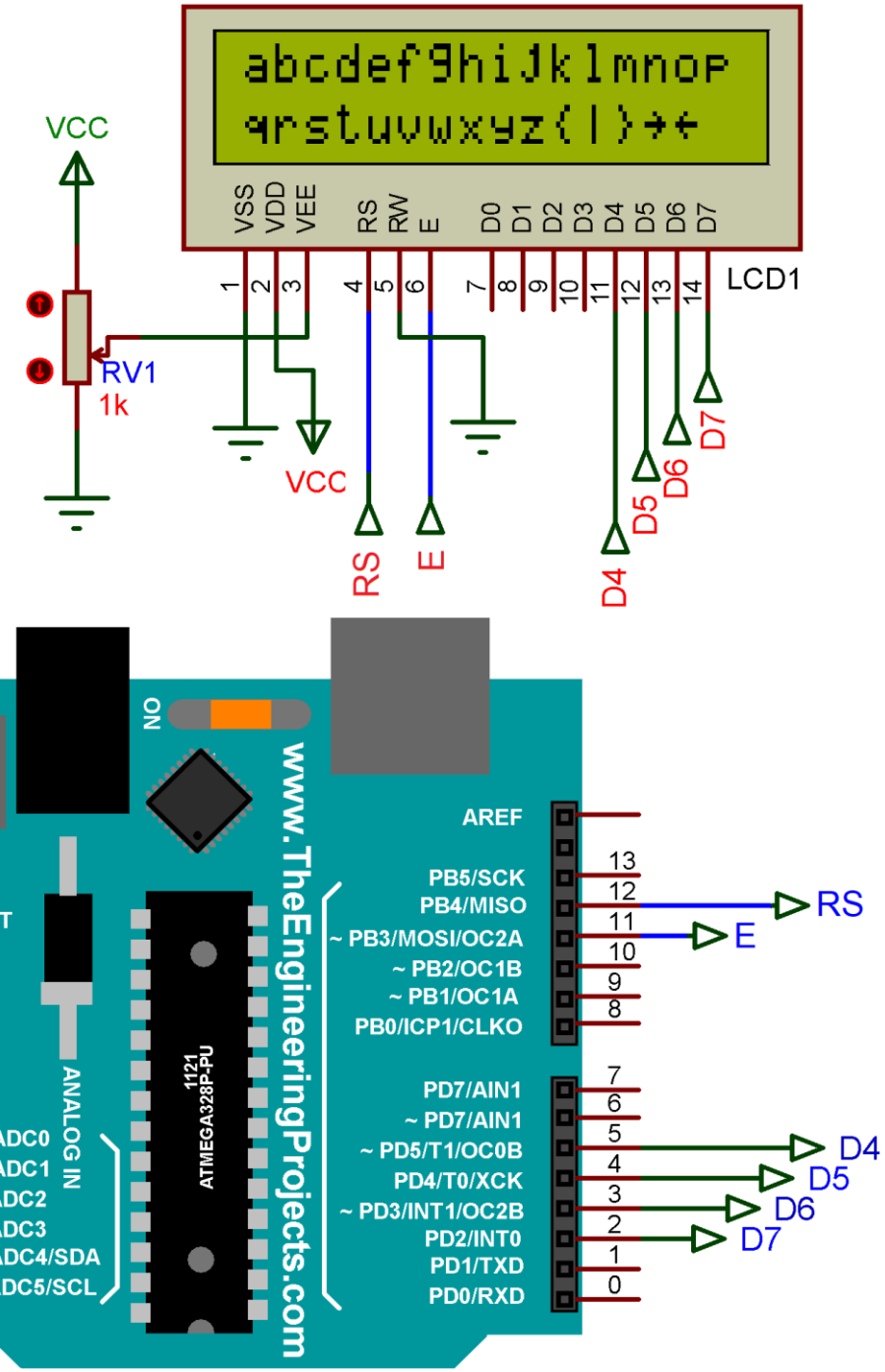
C → column-number
R → row-number



- By default, the starting location is (0,0). So, if you call `print( )` without first changing the cursor location, the text starts in the top-left corner

Example: Write an Arduino Sketch to display ASCII code on 16x2 LCD display using 4-bit mode (data pins D4,D5, D6 & D7)

- 1st ASCII code is printed at location (0,0), the 2nd ASCII code is printed at location (1,0) and so on
- The ASCII code shall be displayed at the entire 32 display characters, make sure that the code pauses for 0.5 second time while printing the ASCII code



```
#include <LiquidCrystal.h> // The code starts out using the included LiquidCrystal Library
const int rs = 12, en = 11, d4 = 5, d5 = 4, d6 = 3, d7 = 2;
// Declare integer variables to assign LCD pins to the corresponding Arduino digital pins
LiquidCrystal lcd(rs, en, d4, d5, d6, d7); // Define 4-bit Mode using d4, d5, d6 & d7
int Rows =0, Cols=0;          // Declare two variables to stor Rows & Cols
int Letter='a';               // Declare a variable to stor ASCII code
void setup() {                // Setup code is performed once
lcd.begin(16, 2);             // Initialize 16X2 LCD display
}
void loop() {
while (Rows<2){               // Enter 1st loop as long as Rows<2
  while (Cols<16){            // Enter 2nd loop as long as Cols<16
  lcd.setCursor(Cols, Rows); // Position the cursor to the desired position
  delay(500);                 // Pause for 0.5 second time
  lcd.write(Letter);          // Print ASCII code on the LCD display
  Cols++;                     // Increment Cols variable by 1
  Letter++;                   // Increment ASCII code using Letter variable
  }                           // Closing curly-bracket for intenal while loop
Cols=0;                       // Reset Cols variable
Rows++;                       // Increment Rows variable by one
}                             // Closing curly-bracket for the external while loop
}                             // Closing curly-bracket for the loop function
```

Page-193